Analysis Design of Algorithm

B.Tech

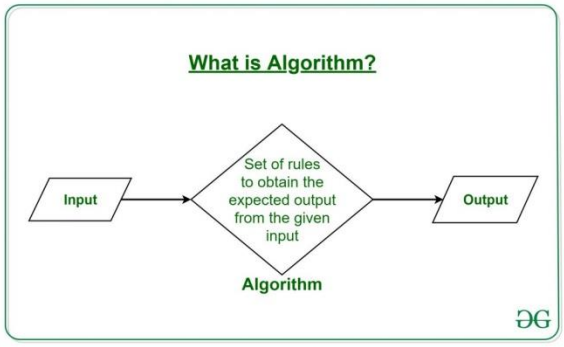
Computer Science Engineering

Semester :- 4th

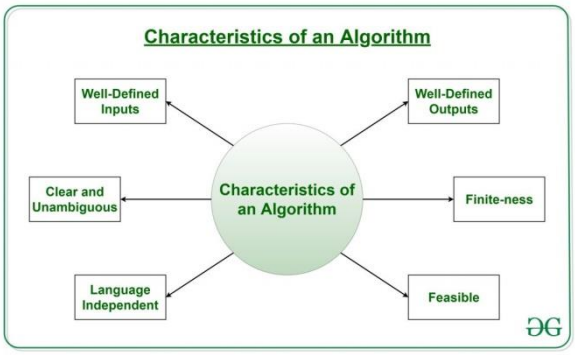
Unit - 1

❖ Algorithm

The word Algorithm means ” A set of finite rules or instructions to be followed in calculations or other problem-solving operations ”



➢ Characteristics of an Algorithm



- **Clear and Unambiguous**: The algorithm should be unambiguous. Each of its steps should be clear in all aspects and must lead to only one meaning.
- **Well-Defined Inputs**: If an algorithm says to take inputs, it should be well-defined inputs. It may or may not take input.
- **Well-Defined Outputs:** The algorithm must clearly define what output will be yielded and it should be well-defined as well. It should produce at least 1 output.
- **Finite-ness:** The algorithm must be finite, i.e. it should terminate after a finite time.
- **Feasible:** The algorithm must be simple, generic, and practical, such that it can be executed with the available resources. It must not contain some future technology or anything.
- **Language Independent:** The Algorithm designed must be language-independent, i.e. it must be just plain instructions that can be implemented in any language, and yet the output will be the same, as expected.
- **Input**: An algorithm has zero or more inputs. Each that contains a fundamental operator must accept zero or more inputs.

- **Output**: An algorithm produces at least one output. Every instruction that contains a fundamental operator must accept zero or more inputs.
- **Definiteness:** All instructions in an algorithm must be unambiguous, precise, and easy to interpret. By referring to any of the instructions in an algorithm one can clearly understand what is to be done. Every fundamental operator in instruction must be defined without any ambiguity.
- **Finiteness:** An algorithm must terminate after a finite number of steps in all test cases. Every instruction which contains a fundamental operator must be terminated within a finite amount of time. Infinite loops or recursive functions without base conditions do not possess finiteness.
- **Effectiveness:** An algorithm must be developed by using very basic, simple, and feasible operations so that one can trace it out by using just paper and pencil.

➢ Properties of Algorithm:

- It should terminate after a finite time.
- It should produce at least one output.
- It should take zero or more input.
- It should be deterministic means giving the same output for the same input case.

➢ Types of Algorithms:

There are several types of algorithms available. Some important algorithms are:

1. Brute Force Algorithm:

It is the simplest approach to a problem. A brute force algorithm is the first approach that comes to finding when we see a problem.

2. Recursive Algorithm:

A recursive algorithm is based on recursion. In this case, a problem is broken into several sub-parts and called the same function again and again.

3. Backtracking Algorithm:

The backtracking algorithm builds the solution by searching among all possible solutions. Using this algorithm, we keep on building the solution following criteria. Whenever a solution fails we trace back to the failure point build on the next solution and continue this process till we find the solution or all possible solutions are looked after.

4. Searching Algorithm:

Searching algorithms are the ones that are used for searching elements or groups of elements from a particular data structure. They can be of different types based on their approach or the data structure in which the element should be found.

5. Sorting Algorithm:

Sorting is arranging a group of data in a particular manner according to the requirement. The algorithms which help in performing this function are called sorting algorithms. Generally sorting algorithms are used to sort groups of data in an increasing or decreasing manner.

6. Hashing Algorithm:

Hashing algorithms work similarly to the searching algorithm. But they contain an index with a key ID. In hashing, a key is assigned to specific data.

7. Divide and Conquer Algorithm:

This algorithm breaks a problem into sub-problems, solves a single sub-problem, and merges the solutions to get the final solution. It consists of the following three steps:

- Divide
- Solve

- Combine

8. Greedy Algorithm:

In this type of algorithm, the solution is built part by part. The solution for the next part is built based on the immediate benefit of the next part. The one solution that gives the most benefit will be chosen as the solution for the next part.

9. Dynamic Programming Algorithm:

This algorithm uses the concept of using the already found solution to avoid repetitive calculation of the same part of the problem. It divides the problem into smaller overlapping subproblems and solves them.

- **Design an Algorithm**

To write an algorithm, the following things are needed as a pre-requisite:

1. The **problem** that is to be solved by this algorithm i.e. clear problem definition.
2. The **constraints** of the problem must be considered while solving the problem.
3. The **input** to be taken to solve the problem.
4. The **output** is to be expected when the problem is solved.
5. The **solution** to this problem is within the given constraints.

- **How to analyze an Algorithm?**

1. Priori Analysis:

"Priori" means "before". Hence Priori analysis means checking the algorithm before its implementation. In this, the algorithm is checked when it is written in the form of theoretical steps.

2. Posterior Analysis:

"Posterior" means "after". Hence Posterior analysis means checking the algorithm after its implementation. In this, the algorithm is checked by implementing it in any programming language and executing it.

- **Algorithm complexity and how to find it?**

The two factors of Algorithm Complexity are:

- **Time Factor**: Time is measured by counting the number of key operations such as comparisons in the sorting algorithm.
- **Space Factor**: Space is measured by counting the maximum memory space required by the algorithm to run/execute.

. **Space Complexity:** The space complexity of an algorithm refers to the amount of memory required by the algorithm to store the variables and get the result. This can be for inputs, temporary operations, or outputs.

How to calculate Space Complexity?

The space complexity of an algorithm is calculated by determining the following 2 components:

- **Fixed Part:** This refers to the space that is required by the algorithm. For example, input variables, output variables, program size, etc.

- **Variable Part:** This refers to the space that can be different based on the implementation of the algorithm. For example, temporary variables, dynamic memory allocation, recursion stack space, etc. Therefore Space complexity **S(P)** of any algorithm P is **S(P) = C + SP(I)**, where C is the fixed part and S(I) is the variable part of the algorithm, which depends on instance characteristic I.

Time Complexity: The time complexity of an algorithm refers to the amount of time required by the algorithm to execute and get the result. This can be for normal operations, conditional if-else statements, loop statements, etc.

How to Calculate, **Time Complexity?**

The time complexity of an algorithm is also calculated by determining the following 2 components:

- **Constant time part:** Any instruction that is executed just once comes in this part. For example, input, output, if-else, switch, arithmetic operations, etc.
- **Variable Time Part:** Any instruction that is executed more than once, say n times, comes in this part. For example, loops, recursion, etc.
  Therefore Time complexity T(P) of any algorithm P is **T(P) = C + TP(I)**, where C is the constant time part and TP(I) is the variable part of the algorithm, which depends on the instance characteristic I.

❖ Asymptotic Notations:

- Asymptotic Notations are mathematical tools that allow you to analyze an algorithm's running time by identifying its behavior as its input size grows.
- This is also referred to as an algorithm's growth rate.
- You can't compare two algorithm's head to head.
- You compare space and time complexity using asymptotic analysis.

There are mainly three asymptotic notations:

1. ***Big-O Notation (O-notation)***
2. ***Omega Notation (Ω-notation)***
3. ***Theta Notation (Θ-notation)***

➢ Big-O Notation (O-notation):

Big-O notation represents the upper bound of the running time of an algorithm. Therefore, it gives the worst-case complexity of an algorithm.
.It is the most widely used notation for Asymptotic analysis.
.It specifies the upper bound of a function.
.The maximum time required by an algorithm or the worst-case time complexity.

Mathematical Representation of Big-O Notation:
$O(g(n))$

➢ Omega Notation (Ω-Notation):

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.

Mathematical Representation of Omega notation :
$\Omega(g(n))$

- **Theta Notation (Θ-Notation):**

*Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the **average-case** complexity of an algorithm.*
.Theta (Average Case) You add the running times for each possible input combination and take the average in the average case.

Mathematical Representation of Theta notation:
Θ (g(n))

❖ Heap

A heap is a complete binary tree, and the binary tree is a tree in which the node can have the utmost two children. A complete binary tree is a binary tree in which all the levels except the last level, i.e., leaf node, should be completely filled, and all the nodes should be left-justified.

- **Heap Sort**

Heap sort is a popular and efficient sorting algorithm. The concept of heap sort is to eliminate the elements one by one from the heap part of the list, and then insert them into the sorted part of the list.

Heapsort is the in-place sorting algorithm.

❖ Heap Sort Algorithm

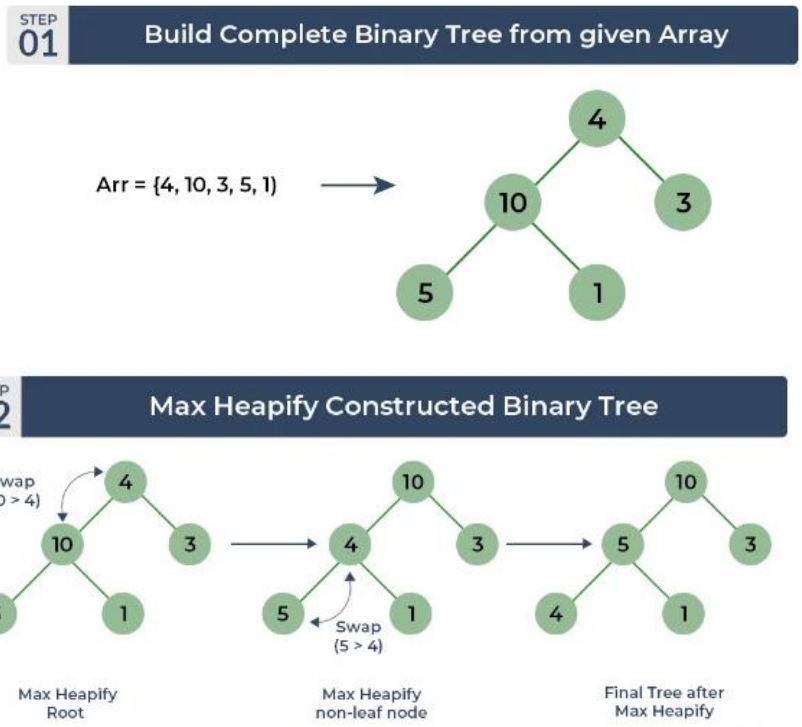
First convert the array into heap data structure using heapify, then one by one delete the root node of the Max-heap and replace it with the last node in the heap and then heapify the root of the heap. Repeat this process until size of heap is greater than 1.

- Build a heap from the given input array.
- Repeat the following steps until the heap contains only one element:
    - Swap the root element of the heap (which is the largest element) with the last element of the heap.
    - Remove the last element of the heap (which is now in the correct position).
    - Heapify the remaining elements of the heap.
- The sorted array is obtained by reversing the order of the elements in the input array.

- **Detailed Working of Heap Sort**

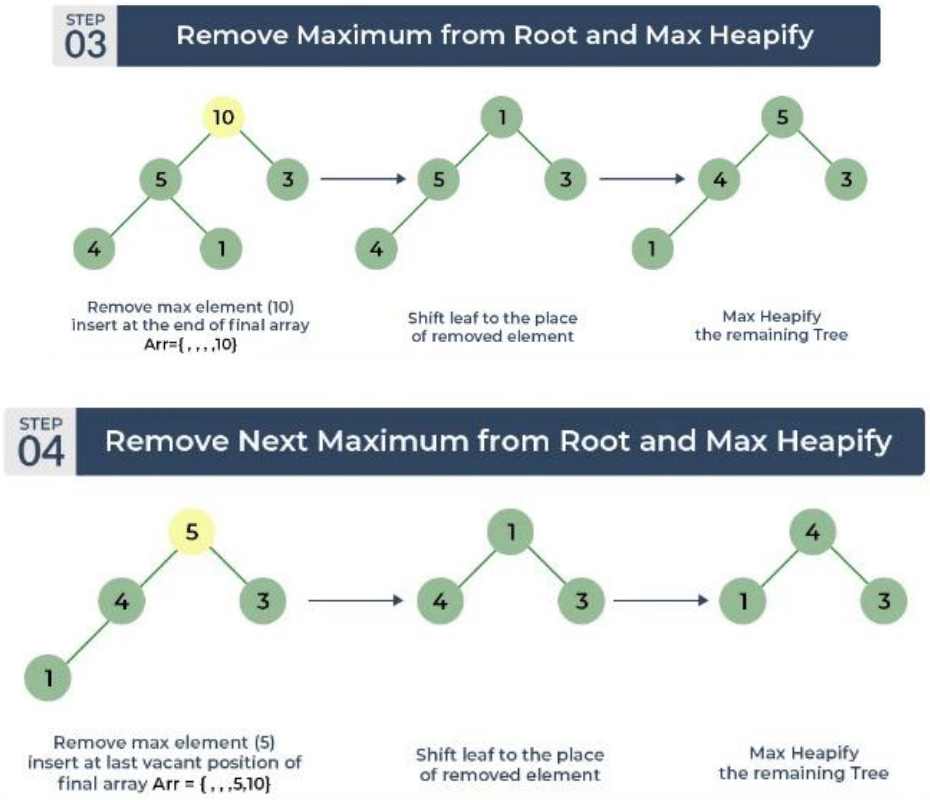
To understand heap sort more clearly, let's take an unsorted array and try to sort it using heap sort. Consider the array: arr[] = {4, 10, 3, 5, 1}.

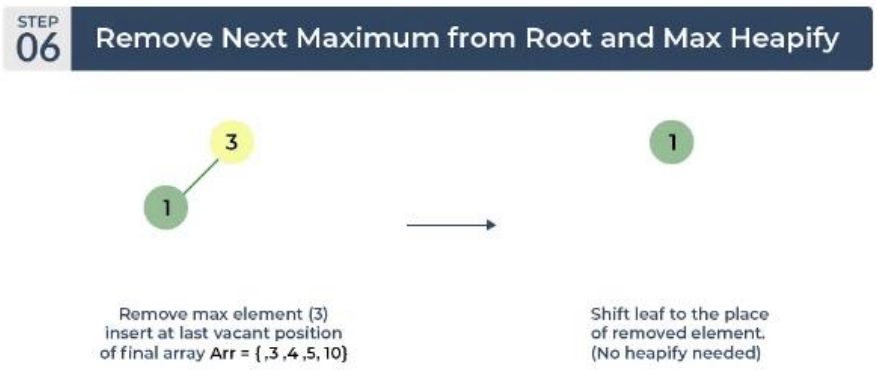
***Build Complete Binary Tree:** Build a complete binary tree from the array.*



- **Perform heap sort:** Remove the maximum element in each step (i.e., move it to the end position and remove that) and then consider the remaining elements and transform it into a max heap.

- Delete the root element **(10)** from the max heap. In order to delete this node, swap it with the last node, i.e. **(1).** After removing the root element, again heapify it to convert it into max heap.
  - Resulted heap and array should look like this:





➢ **Complexity Analysis of Heap Sort**

Time Complexity: O(N log N)
Auxiliary Space: O(log n), due to the recursive call stack. However, auxiliary space can be O(1) for iterative implementation.

➢ **Advantages of Heap Sort:**

- **Efficient Time Complexity:** Heap Sort has a time complexity of O(n log n) in all cases. This makes it efficient for sorting large datasets. The **log n** factor comes from the height of the binary heap, and it ensures that the algorithm maintains good performance even with a large number of elements.
- **Memory Usage –** Memory usage can be minimal because apart from what is necessary to hold the initial list of items to be sorted, it needs no additional memory space to work
- **Simplicity –** It is simpler to understand than other equally efficient sorting algorithms because it does not use advanced computer science concepts such as recursion.

➢ **Disadvantages of Heap Sort:**

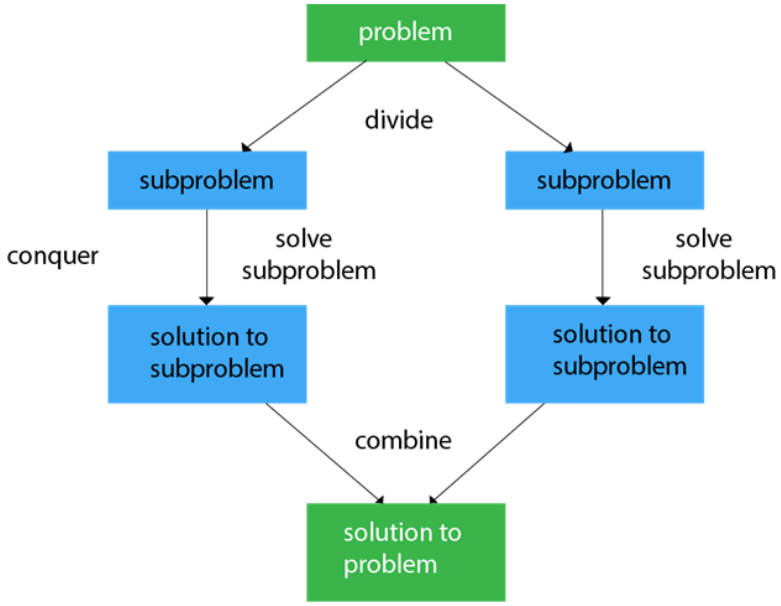
- **Costly**: Heap sort is costly.
- **Unstable**: Heap sort is unstable. It might rearrange the relative order.
- **Efficient:** Heap Sort is not very efficient when working with highly complex data.

❖ Divide And Conquer

Divide and Conquer is an algorithmic pattern. In algorithmic methods, the design is to take a dispute on a huge input, break the input into minor pieces, decide the problem on each of the small pieces, and then merge the piecewise solutions into a global solution. This mechanism of solving the problem is called the Divide & Conquer Strategy.

This technique can be divided into the following three parts:

1. **Divide:** This involves dividing the problem into smaller sub-problems.
2. **Conquer:** Solve sub-problems by calling recursively until solved.
3. **Combine:** Combine the sub-problems to get the final solution of the whole problem.



- **Examples:** The specific computer algorithms are based on the Divide & Conquer approach

1. Maximum and Minimum Problem
2. Binary Search
3. Sorting (merge sort, quick sort)
4. Tower of Hanoi.

- **Fundamental of Divide & Conquer Strategy**:
  There are two fundamental of Divide & Conquer Strategy:

1. Relational Formula
2. Stopping Condition

1. Relational Formula: It is the formula that we generate from the given technique. After generation of Formula we apply D&C Strategy, i.e. we break the problem recursively & solve the broken subproblems.

2. Stopping Condition: When we break the problem using Divide & Conquer Strategy, then we need to know that for how much time, we need to apply divide & Conquer. So the condition where the need to stop our recursion steps of D&C is called as Stopping Condition.

- The following are some standard algorithms that follow Divide and Conquer algorithm.

1. **Quicksort** is a sorting algorithm. The algorithm picks a pivot element and rearranges the array elements so that all elements smaller than the picked pivot element move to the left side of the pivot, and all greater elements move to the right side. Finally, the algorithm recursively sorts the sub arrays on the left and right of the pivot element.

2. **Merge Sort** is also a sorting algorithm. The algorithm divides the array into two halves, recursively sorts them, and finally merges the two sorted halves.
3. **Closest Pair of Points** It is a problem of computational geometry. This algorithm emphasizes finding out the closest pair of points in a metric space, given n points, such that the distance between the pair of points should be minimal.

4. **Strassen's Algorithm** It is an algorithm for matrix multiplication, which is named after Volker Strassen. It has proven to be much faster than the traditional algorithm when works on large matrices.

5. **Cooley–Tukey Fast Fourier Transform (FFT) algorithm** The Fast Fourier Transform algorithm is named after J. W. Cooley and John Turkey. It follows the Divide and Conquer Approach and imposes a complexity of O(nlogn).
6. **Karatsuba algorithm for fast multiplication** It is one of the fastest multiplication algorithms of the traditional time, invented by Anatoly Karatsuba in late 1960 and got published in 1962. It multiplies two n-digit numbers in such a way by reducing it to at most single-digit.

- **Advantages of Divide and Conquer**
  - Divide and Conquer tend to successfully solve one of the biggest problems, such as the Tower of Hanoi, a mathematical puzzle. It is challenging to solve complicated problems for which you have no basic idea, but with the help of the divide and conquer approach, it has lessened the effort as it works on dividing the main problem into two halves and then solve them recursively. This algorithm is much faster than other algorithms.
  - It efficiently uses cache memory without occupying much space because it solves simple subproblems within the cache memory instead of accessing the slower main memory.
  - It is more proficient than that of its counterpart Brute Force technique.
  - Since these algorithms inhibit parallelism, it does not involve any modification and is handled by systems incorporating parallel processing.

Disadvantages of Divide and Conquer

- Since most of its algorithms are designed by incorporating recursion, so it necessitates high memory management.
- An explicit stack may overuse the space.
- It may even crash the system if the recursion is performed rigorously greater than the stack present in the CPU.

❖ Max - Min Problem

Problem: Analyze the algorithm to find the maximum and minimum element from an array.

Algorithm
Input: Array a[] of n elements
Output: Maximum and minimum elements of a[]

```
FindMaxMin(a[])

max = a[1]
min = a[1]

For i = 2 to n do
   If a[i] > max then
      max = a[i]
   If a[i] < min then
      min = a[i]

Return (max, min)
```

- **Analysis:**

Method 1: if we apply the general approach to the array of size n, the number of comparisons required are 2n-2.

Number of comparisons requires applying general approach on n elements = (n-1) + (n-1) = 2n-2

Method-2: In another approach, we will divide the problem into sub-problems and find the max and min of each group, now max. Of each group will compare with the only max of another group and min with min.

Let n = is the size of items in an array

Let T (n) = time required to apply the algorithm on an array of size n. Here we divide the terms as T(n/2).

2 here tends to the comparison of the minimum with minimum and maximum with maximum as in above example.

$T(n) = T(n/2)+T(n/2)+2$

$T(n) = 2T(n/2)+2 \quad \rightarrow$ Eq (i)

T (2) = 1, time required to compare two elements/items. (Time is measured in units of the number of comparisons)

As we put n= n/2

$T(n/2) = 2T(n/2^2)+2 \quad \rightarrow$ Eq (ii)

Put eq (ii) in eq (i)

$T(n)= 2[2T(n/2^2)+2]+2$

$= 2^2T(n/2^2) + 2^2 + 2$

Similarly, apply the same procedure recursively on each sub problem or anatomy.

{Use recursion means, we will use some stopping condition to stop the algorithm}

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + 2^i + 2^{i-1} + \ldots\ldots\ldots\ldots\ldots\ldots + 2 \quad \ldots\ldots \text{(Eq. 3)}$$

Recursion will stop, when $n/2^i = 2$ (Eq. 4)

Put Eq 4 into Eq 3

Number of comparisons requires applying the divide and conquering algorithm on n elements/items = (3n/2)-2

Analysis: suppose we have the array of size 8 elements.

Method1: requires (2n-2), (2x8)-2=14 comparisons

Method2: requires {(3X8)/2 }-2 = 10 COMPARISIONS

It is evident; we can reduce the number of comparisons (complexity) by using a proper technique.

❖ Binary Search

1. In Binary Search technique, we search an element in a sorted array by recursively dividing the interval in half.

2. Firstly, we take the whole array as an interval.

3. If the Pivot Element (the item to be searched) is less than the item in the middle of the interval, We discard the second half of the list and recursively repeat the process for the first half of the list by calculating the new middle and last element.

4. If the Pivot Element (the item to be searched) is greater than the item in the middle of the interval, we discard the first half of the list and work recursively on the second half by calculating the new beginning and middle element.

5. Repeatedly, check until the value is found or interval is empty.

Binary Search Method.

Index	0	1	2	3	4	5	6	7
Value	8	13	17	26	44	56	88	97

first, mid, last.

mid = (first + last) / 2

= (0+7) / 2

= 7/2 = 3.5 { .5 को avoid कर देते हैं }.

if (88 < 26) → no.

88 > 26 → yes that means 88 is present at the Right side of 26 (mid)

now take Right Sub array.

4	5	6	7
44	56	88	97

first, mid, last

$\frac{4+7}{2} = \frac{11}{2} = 5.5 = 5$ ← mid

88 < 56 → no.

88 > 5.6 → yes that means 88 is at the Right side of the array.

6	7
88	97

mid → first, last.

$\frac{6+7}{2} = \frac{13}{2} = 6.5$.

88 = 88 → the given index is A[6].

Binary Search:

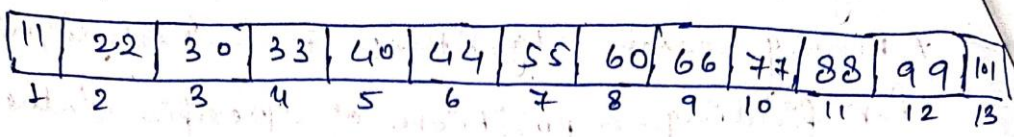
→ A Binary Search algorithm is a Technique for finding a position of specific value with in a sorted array.

→ It is only apply on sorted array.

Algorithm.

```
{
    low = 1;                    // Where low, high,
    high = n;                       mid will be int.
While (low <= high)
  {
    mid = (low + high)/2;
    if (A[mid] == x)
        return mid;
    else if (~~A[mid]~~ x < A[mid])
        high = mid - 1;
    else
        low = mid + 1;
  }
return 1;
```

Example :-



$x = 40$

1. initialize low = 1, high = 13

$1 <= 13 \rightarrow$ True

$mid = \frac{1+13}{2} = \frac{14}{2} = 7.$

$A[mid] = 55$ which is $\neq 40$

is $x(40) < A[mid](55)$

true

then

high = mid − 1

= 7 − 1 = 6

now ⟹ high = 6

low = 1

is $1 <= 44 \rightarrow$ true

$mid \frac{1+6}{2} = \frac{7}{2} = 3.5.$

mid = 3.

$A[mid] = 30 \neq 40$

is $x(40) < A[mid](30) \rightarrow$ false

then

low = mid + 1.

= 3 + 1 = 4

now

low = 4

high = 6

low < = high → true

then

mid = ⌊ (4+6)/2 ⌋ = 10/2 = 5

mid = 5

x = = A [mid]

40 = 40

so the searching element is at A [5] search is successful.

❖ Merge Sort

Merge sort is yet another sorting algorithm that falls under the category of Divide and Conquer technique. It is one of the best sorting techniques that successfully build a recursive algorithm.

Divide and Conquer Strategy

In this technique, we segment a problem into two halves and solve them individually. After finding the solution of each half, we merge them back to represent the solution of the main problem.

Suppose we have an array **A**, such that our main concern will be to sort the subsection, which starts at index **p** and ends at index **r**, represented by **A[p..r]**.

Divide

If assumed **q** to be the central point somewhere in between **p** and **r**, then we will fragment the subarray **A[p..r]** into two arrays **A[p..q]** and **A[q+1, r]**.

Conquer

After splitting the arrays into two halves, the next step is to conquer. In this step, we individually sort both of the subarrays **A[p..q]** and **A[q+1, r]**. In case if we did not reach the base situation, then we again follow the same procedure, i.e., we further segment these subarrays followed by sorting them separately.

Combine

As when the base step is acquired by the conquer step, we successfully get our sorted subarrays **A[p..q]** and **A[q+1, r]**, after which we merge them back to form a new sorted array **[p..r]**.

- **Merge Sort algorithm**

The MergeSort function keeps on splitting an array into two halves until a condition is met where we try to perform MergeSort on a subarray of size 1, i.e., **p == r**.

And then, it combines the individually sorted sub arrays into larger arrays until the whole array is merged.

- **ALGORITHM-MERGE SORT**

1. If p**<r**
2. Then q → (p+ r)/2
3. MERGE-SORT (A, p, q)
4. MERGE-SORT (A, q+1,r)
5. MERGE (A, p, q, r)

Here we called **MergeSort(A, 0, length(A)-1)** to sort the complete array.

As you can see in the image given below, the merge sort algorithm recursively divides the array into halves until the base condition is met, where we are left with only 1 element in the array. And then, the merge function picks up the sorted sub-arrays and merge them back to sort the entire array.

Example:-

Consider the following example of an unsorted array, which we are going to sort with the help of the Merge Sort algorithm.

A= (36,25,40,2,7,80,15)

Step1: The merge sort algorithm iteratively divides an array into equal halves until we achieve an atomic value. In case if there are an odd number of elements in an array, then one of the halves will have more elements than the other half.

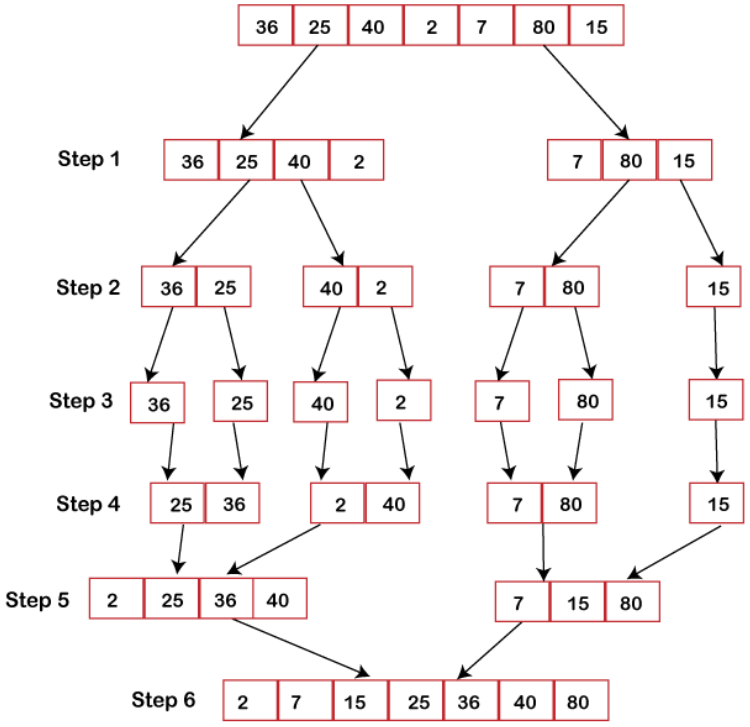
Step2: After dividing an array into two subarrays, we will notice that it did not hamper the order of elements as they were in the original array. After now, we will further divide these two arrays into other halves.

Step3: Again, we will divide these arrays until we achieve an atomic value, i.e., a value that cannot be further divided.

Step4: Next, we will merge them back in the same way as they were broken down.

Step5: For each list, we will first compare the element and then combine them to form a new sorted list.

Step6: In the next iteration, we will compare the lists of two data values and merge them back into a list of found data values, all placed in a sorted manner.

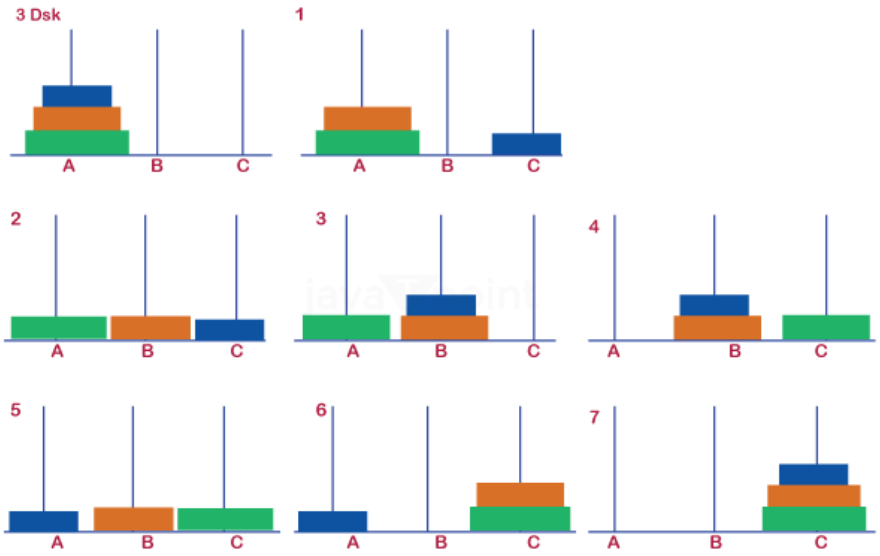


❖ Tower of Hanoi

1. It is a classic problem where you try to move all the disks from one peg to another peg using only three pegs.

2. Initially, all of the disks are stacked on top of each other with larger disks under the smaller disks.

3. You may move the disks to any of three pegs as you attempt to relocate all of the disks, but you cannot place the larger disks over smaller disks and only one disk can be transferred at a time.



This problem can be easily solved by Divide & Conquer algorithm

In the above 7 step all the disks from peg A will be transferred to C given Condition:

1. Only one disk will be shifted at a time.
2. Smaller disk can be placed on larger disk.

Let T (n) be the total time taken to move n disks from peg A to peg C

1. Moving n-1 disks from the first peg to the second peg. This can be done in T (n-1) steps.
2. Moving larger disks from the first peg to the third peg will require first one step.
3. Recursively moving n-1 disks from the second peg to the third peg will require again T (n-1) step.

So, total time taken T (n) = T (n-1)+1+ T(n-1)

METHOD NAME	EQUATION	STOPPING CONDITION	COMPLEXITIES
1.Max&Min	$T(n) = 2T\left(\frac{n}{2}\right)+2$	T(2)=1	$T(n) = \frac{3N}{2} - 2$
2.Binary Search	$T(n) = T\left(\frac{n}{2}\right) + 1$	T(1)=1	T(n)=log n
3.Merge Sort	$T(n)=2T\left(\frac{n}{2}\right)+n$	T(1)=0	T(n)=nlog n
4. Tower of Hanoi	T(n)=2T(n-1)+1	T(1)=1	$T(n)=2^n-1$

❖ Quick sort

It is an algorithm of Divide & Conquer type.

Divide: Rearrange the elements and split arrays into two sub-arrays and an element in between search that each element in left sub array is less than or equal to the average element and each element in the right sub- array is larger than the middle element.

Conquer: Recursively, sort two sub arrays.

Quick Sort-

(35)	50	15	25	80	20	90	45
↑	↑						↑
Pivot	P						Q

We take two pointers P and Q.
P will move R.H.S. and stop it Reaches greater than Pivot
Q will move L.H.S., stops when Lesser than Pivot

Why we done this? bcoz pivot value Reach at correct- place

← Less Value | Pivot | → greater Values.

now check first value: for P.

50 > 35 (Pivot) → true P, stop.

P will not move

check first value for: Q.

45 < 35 → no.

90 < 35 → no

20 < 35 → yes, then Q will shift- to the 20. Q. stop.

(35)	50	15	25	80	20	90	45
	↑				↑		
	P				Q		

now check that- P, Q have cross each other or not if not then swap them.

(35)	20	15	25	80	50	90	45
	↑				↑		
	P				Q		

now check the Value for p.

20 > 35 → no

15 > 35 → no

25 > 35 → no

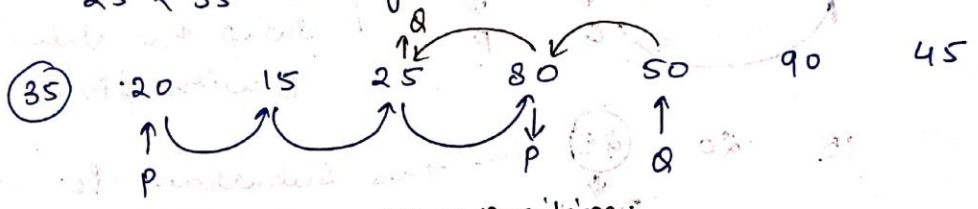
80 > 35 → yes, p stop.

now check the value for q.

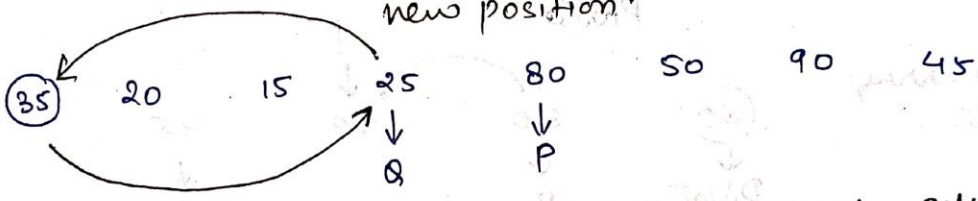
50 < 35 → no

80 < 35 → no

25 < 35 → yes, q. stop.



new position:



→ we see that p and q cross each other in that condition we have to swap the pivot element with q.

25 20 15 [35] 80 50 90 45

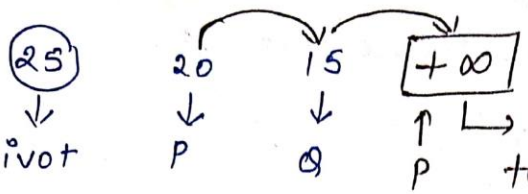
→ we can see that our pivot value is at right place.

→ now array will devide into two subarrays.

25 20 15 80 50 90 45

→ we have to apply same algo to the rest pair.

Subarray I



This have to be with P to stop the P pointer → as it compare it will give the garbage value and stops.

check for P

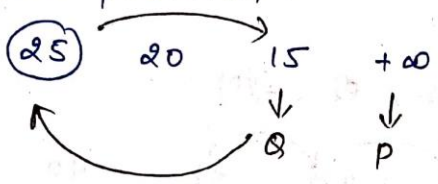
20 > 25 → no

15 > 25 → no P stop.

check for Q.

15 < 25 → yes

New position.

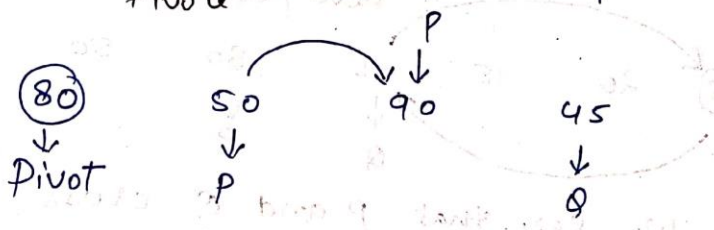


} P and Q swap each other so swap the value of Q with pivot.

15 20 (25) ← this subarray is sorted.

Pivote

Subarray II



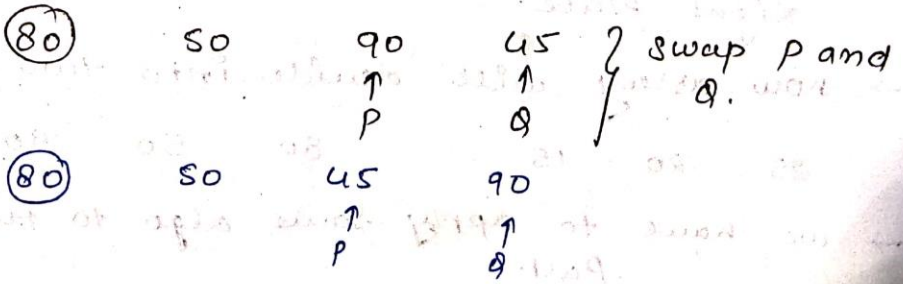
check the value for P.

50 > 80 → no

90 > 80 → yes. P stop.

check the value for Q.

45 < 80 → yes. Q stop.



} swap P and Q.

Check for P

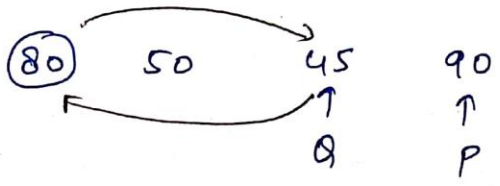
45 > 80 → no

90 > 80 → yes. P stop.

Check for Q

45 < 80 → yes., Q stop.

} as we can see P and Q cross each other so swap pivot with Q.



45 50 80 90. → and we can see that the array is sorted now.

Quick Sort Algorithm :-

The Quick Sort Algorithm is performed using following two important functions -

Quick and partition.

Algorithm: Unsorted Array. first elem. last elem.

```
Quick ( A[0.... n-1], low, high).
{
if ( low < high) // that means there are more than 2 values are there into the array.
then                then we have to partition the Array.
    m = partition (A[low ... high]
          └→ // we have to partition the array into two parts where
    Quick (A[low ... m-1]) // left subarray will contain lesser values.
    Quick (m+1 .... high) // and Right subarray will contain heighest values.
}.
```

So this Quick function is recursively called for dividing the list / array into two sublist

and Partition function is used for arranging the elements into ascending order.

Here is the pseudo code for it.

```
Partition (A[low ... high])
{
   Pivot = A[low]
    i = low (first element of array)
    j = high (last element of array).
  While (i<j) do ← index position
  {
    While (A[i] < pivot) do // by this we get
          i = i+1;          the new value of i
    While (A[j] >=Pivot) do // by this we get
          j = j-1;          the new value of j
  // there will be one condition where i
     will with new value and j will stop.
     with new value and did not move
     further. then we have to check.
       the condn.
  if (j<i) then <---> index posit.
     Swap (A[i], A[j])

  }
  Swap (A[low], A[j]) // when i cross j
  return j;              swap the value of A[j]
                         with A[low] (pivot)
}
```

↓

This will indicate, from where the list will be partitioned.

Strassen's Matrix Multiplication

Suppose we want to multiply two matrices. A and B each of size N i.e.

$$C = A \times B$$

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Traditional Method:

C_{11} = 1st Row 1st column.

$= (A_{11} \cdot B_{11}) + (A_{12} \cdot B_{21})$

C_{12} = 1st Row 2nd column.

$(A_{11} \cdot B_{12}) + (A_{12} \cdot B_{22})$

C_{21} = 2nd Row 1st column.

$(A_{21} \cdot B_{11})$ $(A_{22} \cdot B_{21})$

C_{22} = 2nd Row 2nd column.

$(A_{21} \cdot B_{12})$ $(A_{22} B_{22})$

Divide and Conqure Strategy

two $n \times n$ Matrix can be Multiplied where $n = 2^K$

if we take $K = 1$, $n = 2$ that means to apply divide and conqure technique minimum Value of $n = 2$.

Means 2×2 Matrix.

Divide and conques approach can be used for implementing strassen's Matrix Multiplication.

1) Divide - Divide Matrics into sub Matrics.

2) Conques - Use a group of matrix Multiply equations.

3) Combine - Multiply sub matrices and get final Result after performing Required adds and subetractions.

formula :-

Digonal element Add (+) * (+) Add.

$P \rightarrow (A_{11} + A_{22})(B_{11} + B_{22})$

$Q \rightarrow B_{11}(A_{21} + A_{22})$

$R \rightarrow A_{11}(B_{12} - B_{22})$

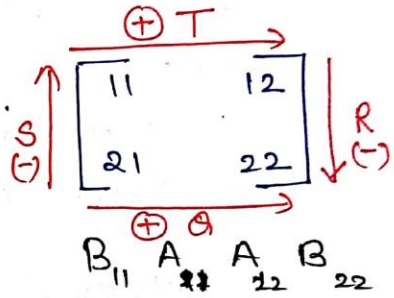
$S \rightarrow A_{22}(B_{21} - B_{11})$

$T \rightarrow B_{22}(A_{11} + A_{12})$

$U \rightarrow$ Only Brackets Value Just swap A → B B → A

$V \rightarrow (A_{21} - A_{11})(B_{11} + B_{12})$

$\rightarrow (B_{21} + B_{22})(A_{12} - A_{22})$



C_{ij} Calculation

$C_{11} - P + S - T + V$

$C_{12} - R + T$ (Trick RAT)

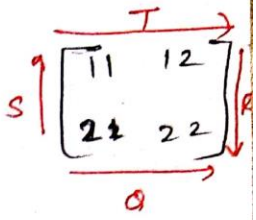
$C_{21} - Q + S$

$C_{22} - P + R - Q + U$

C_{11} and C_{22} start with P.

2. Multiply the matrices using Strassen's Matrix Multiplication

$$A = \begin{bmatrix} 1 & 3 \\ 7 & 5 \end{bmatrix} \qquad B = \begin{bmatrix} 6 & 7 \\ 7 & 8 \end{bmatrix}$$



$P \rightarrow (A_{11}+A_{22})(B_{11}+B_{22}) \Rightarrow (1+5)(6+8) \Rightarrow 6 \times 14 \Rightarrow 84$

$Q \rightarrow B_{11}(A_{21}+A_{22}) \Rightarrow 6(7+5) \Rightarrow 6 \times 12 \Rightarrow 72$

$R \rightarrow A_{12}(B_{12}-B_{22}) \Rightarrow 1(7-8) \Rightarrow -1$

$S \rightarrow A_{22}(B_{21}-B_{11}) \Rightarrow 5(7-6) \Rightarrow 5$

$T \rightarrow B_{22}(A_{11}+A_{12}) \Rightarrow 8(1+3) \Rightarrow 8 \times 4 \Rightarrow 32$

$U \rightarrow (A_{21}-A_{11})(B_{11}+B_{12}) \Rightarrow (7-1)(6+7) \Rightarrow 6 \times 13 \Rightarrow 78$

$V \rightarrow (B_{21}+B_{22})(A_{12}-A_{22}) \Rightarrow (7+8)(3-5) \Rightarrow 15 \times -2 \Rightarrow -30$

$C_{11} \rightarrow P+S-T+V \Rightarrow 84+5-32+(-30) \Rightarrow 89-62 \Rightarrow 27$

$C_{12} \rightarrow R+T \Rightarrow -1+32 \Rightarrow 31$

$C_{21} \rightarrow Q+S \Rightarrow 72+5 \Rightarrow 77$

$C_{22} \rightarrow P+R-Q+U \Rightarrow 84+(-1)-72+78 \Rightarrow 84-73+78 \Rightarrow 162-73 \Rightarrow 89$

try yourself.

$$A = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} \qquad B = \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix} \qquad C = ?$$

Ans $C = \begin{bmatrix} 12 & 11 \\ 22 & 21 \end{bmatrix}$